Multi-Agent Disruption Replanning for Manufacturing Supply Chains

Project Report

Aditya Sharma (PRN 24070126016)
Amaan Saxena (PRN 24070126029)
Division A1, B.Tech Artificial Intelligence and Machine Learning
Symbiosis Institute of Technology, Pune

Code: github.com/amaansaxena/multi-agent-disruption-replanner

1. Abstract

If a supplier gives notice that a part will be delivered late, the factory planner has to figure out what customer orders will be missed and what recovery strategy to choose, and all this is done manually on spreadsheets. This research aims at creating a proof of concept where four cooperative agents perform this task. The Planner agent processes the notice, the Impact Analyst discovers the orders to miss the deadline, the Replanner chooses the optimal recovery strategy using optimization algorithms and writes supplier and customer messages, and the Critic recalculates all numbers from scratch based on original data..

The agents are controlled using LangGraph; the plan is obtained using PuLP; while a Streamlit application will allow the individual to notify about the delay and make decisions. In 20 test cases conducted on synthetic data, all 20 terminated correctly, the optimized plan was cheaper than manually done first come first served correction in 12 cases that required a plan (the average savings were 19%), and the Critic detected both introduced errors.

2. Problem Statement

Today a planner handles a late supplier in six manual steps, and steps 2 to 6 take the most time:

1. The alert arrives as an email or portal note, and someone retypes it.
2. The planner traces the part through the bill of materials by hand to find affected orders.
3. Stock, backup suppliers and expedite options are looked up one at a time.
4. A plan is chosen, often the first workable one rather than the cheapest.
5. Emails go to the supplier and customers, with numbers copied across by hand.
6. If a second delay lands, the planner goes back to step 2.

No one verifies the numbers before sending out the proposal. This procedure will require three things: identify the least expensive solution to recovery instead of just the first solution, verify the numbers independently of the data collected, and maintain a human in control of the final decision.

3. Domain and Dataset

Domain = Supply Chain Planning in manufacturing. The plant produces a number of products using a number of parts; each of the parts has one primary supplier and one or two secondary suppliers. Each order has a deadline and also a per-day cost associated with being late. One delayed part will delay production of all products using the part and all orders pending for these products.

Key terms. BOM: What parts make up which products and in what quantity. Expedited shipment: Additional charges for getting a shipment earlier than normal. Backup vendor: Alternative supplier of parts, often at a higher cost. Shortfall: The difference between what was ordered and what was received.

All data is synthetic and generated by our own script (data/generate_data.py) with a fixed random seed (42), so every run sees the same data. No real company data or public dataset is used.

File	One row is	Notes
suppliers.csv	a supplier offering a part	4 suppliers, 8 parts; price, lead time, capacity, main or backup
bom.csv	a part used in a product	3 products; quantity of each part per unit
inventory.csv	stock of one part	Covers only 15 to 50% of demand; one part has no record, treated as zero
orders.csv	a customer order	25 orders (21 open); due day 8 to 28, penalty 200 to 900 per day
scenarios.json	a test alert	20 test cases covering normal, edge and failure situations

4. Proposed Solution

The process involves the distribution of the task among four agents, each of whom is responsible for only one function, accompanied by a final decision made by humans. The numbers come from the program and the solver and not the language model.

Agent	What it does	Takes in	Hands out
Planner	Reads the alert, rejects bad ones, sets the task order, runs the retry loop and escalates when needed	Raw alert text	Disruption record and run log
Impact Analyst	Traces the late part to products and open orders, then finds the shortage and days late	Disruption record, BOM, stock, orders	Impact report per order
Replanner	Picks the cheapest recovery mix with an optimization model, runs a first-come-first-served baseline, drafts messages	Impact report, supplier options, Critic feedback	Recovery plan, total cost, baseline cost, messages
Critic	Recomputes the cost from raw data and checks the rules	Plan, messages, source data	Pass or fail, with issues

It is the Critic that analyzes the raw data, not the Replanner's own totals; therefore, an erroneous figure cannot verify itself. Architecture and data flow can be seen in Figure 1 below..

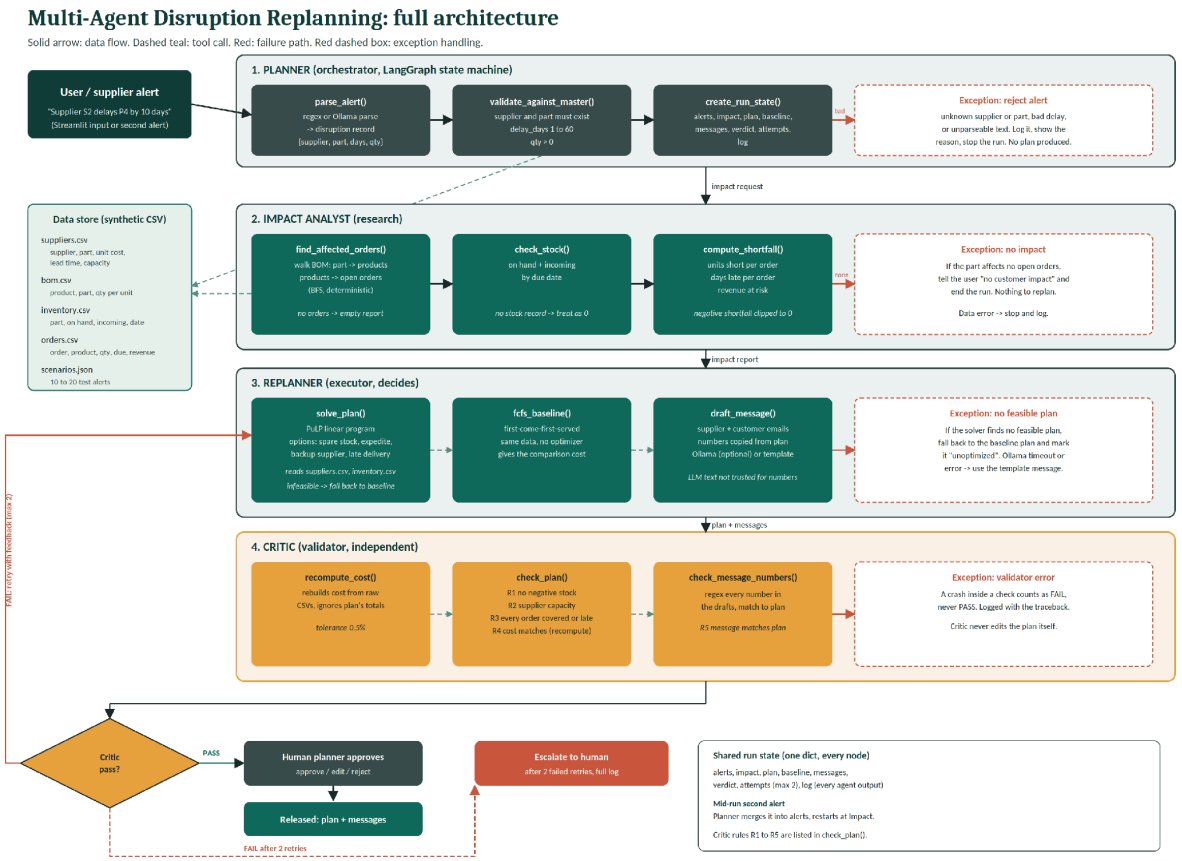


Figure 1. Full architecture: agents, their tools, the data store, failure paths and exception handling

5. Orchestration and Exception Handling

Run states for all agents include alerts, impact report, plan, baseline, messages, Critic evaluation, retry number, and log. In the normal process, the Planner evaluates the alert, followed by the Impact Analyst, Replanner, and Critic, in that order. If the Critic clears the plan, the human planner gets the plan. However, if it fails, the plan returns to the Replanner with Critic feedback, up to two tries, after which it is escalated to a human agent with the full log. Should there be a second alert during the run, the Planner incorporates it and goes back to the Impact Analyst.

Situation	What happens
Bad alert	Unknown supplier or part, delay outside 1 to 60 days, or unreadable text. The run stops and says why.
No impact	The part affects no open orders, or the delay is absorbed by stock and spare time. Nothing is replanned.
Missing stock record	Treated as zero, so the shortage shows up instead of hiding.
No feasible plan	Fall back to the baseline plan and mark it unoptimized.
Critic finds a plan error	Solve again; on a second failure switch to the baseline plan.
Critic finds only a message error	Keep the plan and redraft only the messages.
Human rejects the plan	Offer the baseline plan instead; a second rejection goes to a person.

Situation	What happens
Critic check crashes	Counts as a fail, never a pass.

Critic rules

- **R1** No negative stock: the plan never uses more stock than is on hand.
- **R2** Supplier quantities stay within capacity, and every source in the plan really exists.
- **R3** Every affected order is fully covered, or listed as late with its delay.
- **R4** The recomputed cost matches the reported cost within a small tolerance.
- **R5** Every number in every message matches the plan.

6. Optimization Model

For each impacted order, the Replanner determines the number of units to be pulled from each resource. There are four different resources: on-hand inventory (free, immediate delivery), late delivery (list price, late delivery), expedited late delivery (35% higher cost, 50% shorter lead-time, constrained to 50% of capacity), and backup supplier (10% higher cost, lead time & capacity).

The model used is a mixed integer linear program using PuLP and CBC optimizer. Number of units are integers and a binary decision variable indicates if any of the sources delivering past their due date is used in the order, as any delayed unit will make the order delayed. The goal is to minimize total parts cost and delay penalties such that all orders are satisfied, source capacity constraints are respected, and rush and wait units combined do not exceed the number in transit.

The comparison benchmark is the first come first served criterion which acts as a proxy for the overwhelmed planner who processes orders by deadlines, uses existing inventory, and then the first rush or back-up supplier capable of delivering the entire remaining quantity on time. It is the default when the optimized schedule fails to work.

7. Implementation

Component	Technology and role
Agent workflow	LangGraph state graph: nodes for each agent, conditional edges for pass, fail, retry, reject and escalate
Optimization	PuLP with the CBC solver
Data handling	pandas, reading four CSV files
User interface	Streamlit app: report a delay, see the plan, approve or reject
Alerts and messages	Rule-based alert reading and template messages, so no number depends on a language model
Testing	Tool tests, agent-path tests, app click-through tests and a 20-case scenario runner

The user selects the late supplier, item, and days late on the app, or loads a pre-created case. A schedule that needs approval is presented in Fig. 2; if nothing happens, 5 orders will be late and will incur a penalty cost of 18,350 while the schedule itself is valued at 9,363, 15% cheaper than the

manual remedy. No order is shipped before the user clicks Approve, while Rejecting will provide the manual remedy.

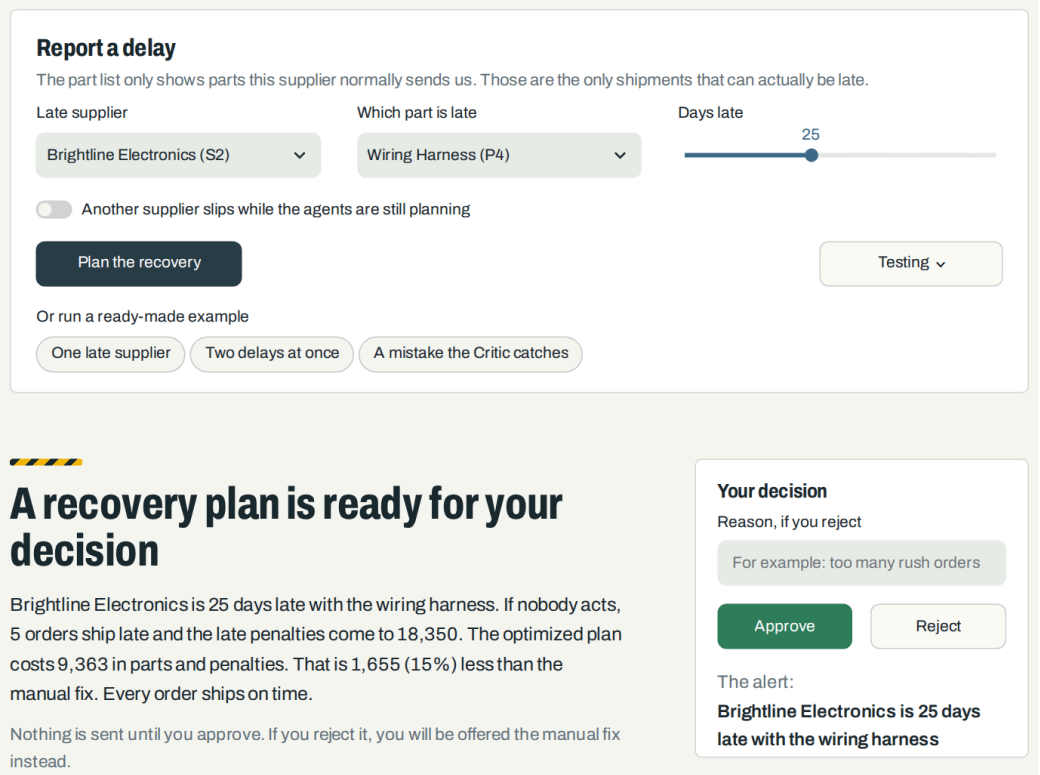


Figure 2. Reporting a delay and deciding on the recovery plan

Figure 3 depicts all the orders that were affected per day, considering two delay scenarios at the same time. Each row represents an individual customer's order, where the line stretches up to the day it was supposed to be delivered and the dot represents the day of its shipment.



Figure 3. Every affected order, day by day, with two delays at once

To prove the Critic works, a mistake can be planted on purpose. When supplier messages always quote one unit too many, the Critic finds 363 where the plan says 362, sends the plan back, fails the rewritten messages again, and after three failures in a row hands the case to a person (Figure 4).

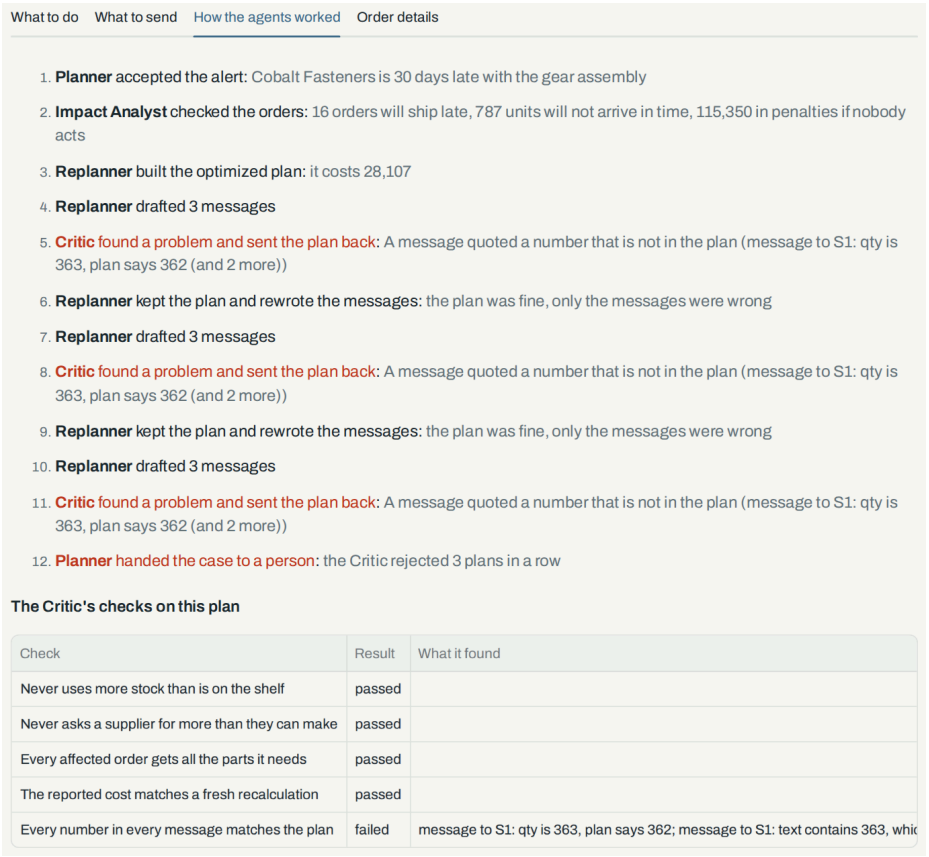


Figure 4. The Critic catches a planted mistake and escalates to a person

8. Results and Validation

Result	Meaning
20 of 20	test cases end the way they should
12 of 12	replans cost less than the manual fix
19%	median saving, ranging from 5% to 80% across cases
2 of 2	planted mistakes caught by the Critic

Cost is parts plus late penalties. Across the 12 replans the optimized plan had 21 late orders against 22 for the manual fix, but in two cases it had more, because a small penalty can cost less than a rush. Figure 5 shows one such case: the plan leaves 5 orders late against 2 for the manual fix, yet costs 8,193 less.

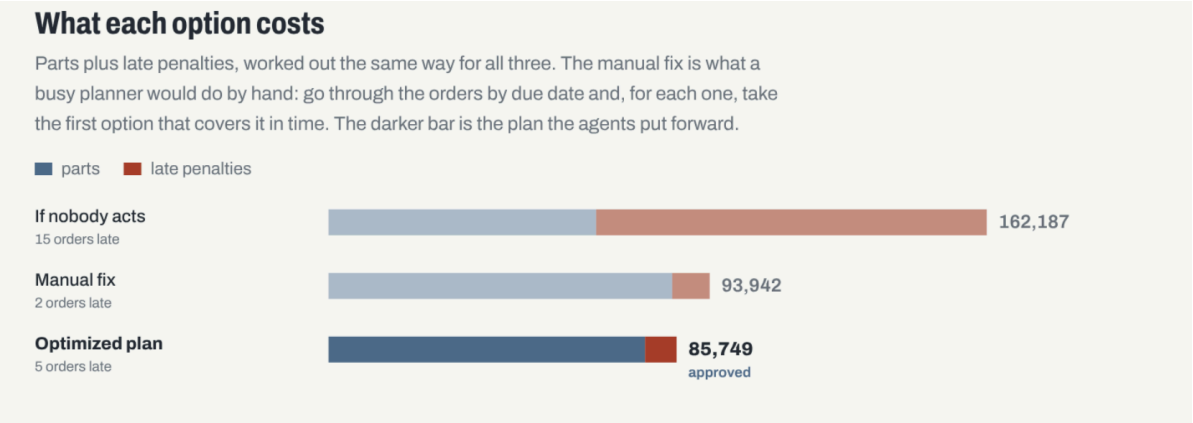


Figure 5. Cost of each option for two delays at once

The median is quoted rather than the total, because two long-delay cases save about 80% and would inflate an overall figure. The manual fix is our own stand-in for a busy planner and the data is synthetic, so these results show that the workflow works, not what a real plant would save.

9. Limitations and Future Work

Known limitations

- The supply chain is small and synthetic, so the results show the workflow, not real savings.
- Costs are modelled linearly, with no uncertainty in how long a delay will last.
- The Critic's rules were written alongside the test cases, so real-world performance would likely be lower.
- Alerts are read with simple rules and messages are templates; this version has no language model in it.

Future work

- Add constraints such as production capacity and multi-tier suppliers.
- Connect real ERP data through connectors.
- Feed supplier replies back into the plan.
- Use a language model to read free-text alerts and draft messages, with the Critic still checking every number.

10. Conclusion

In particular, the solution to the late-supplier scenario is an example where the problem is decomposed into well-defined agent tasks: detecting the alert, assessing its effect, optimizing the response, and validating the optimization independently, with a human agent making the final decision. The separation between execution and verification found all the deliberate errors in the problem, the optimizer outperformed a human baseline on every single replanning case, and the system promotes the problem to a human rather than guesswork.

References

[1] LangGraph documentation, LangChain. https://langchain-ai.github.io/langgraph/

[2] PuLP: a linear programming toolkit for Python. https://coin-or.github.io/pulp/

[3] Streamlit documentation. https://docs.streamlit.io/

[4] Project repository. https://github.com/amaansaxena/multi-agent-disruption-replanner